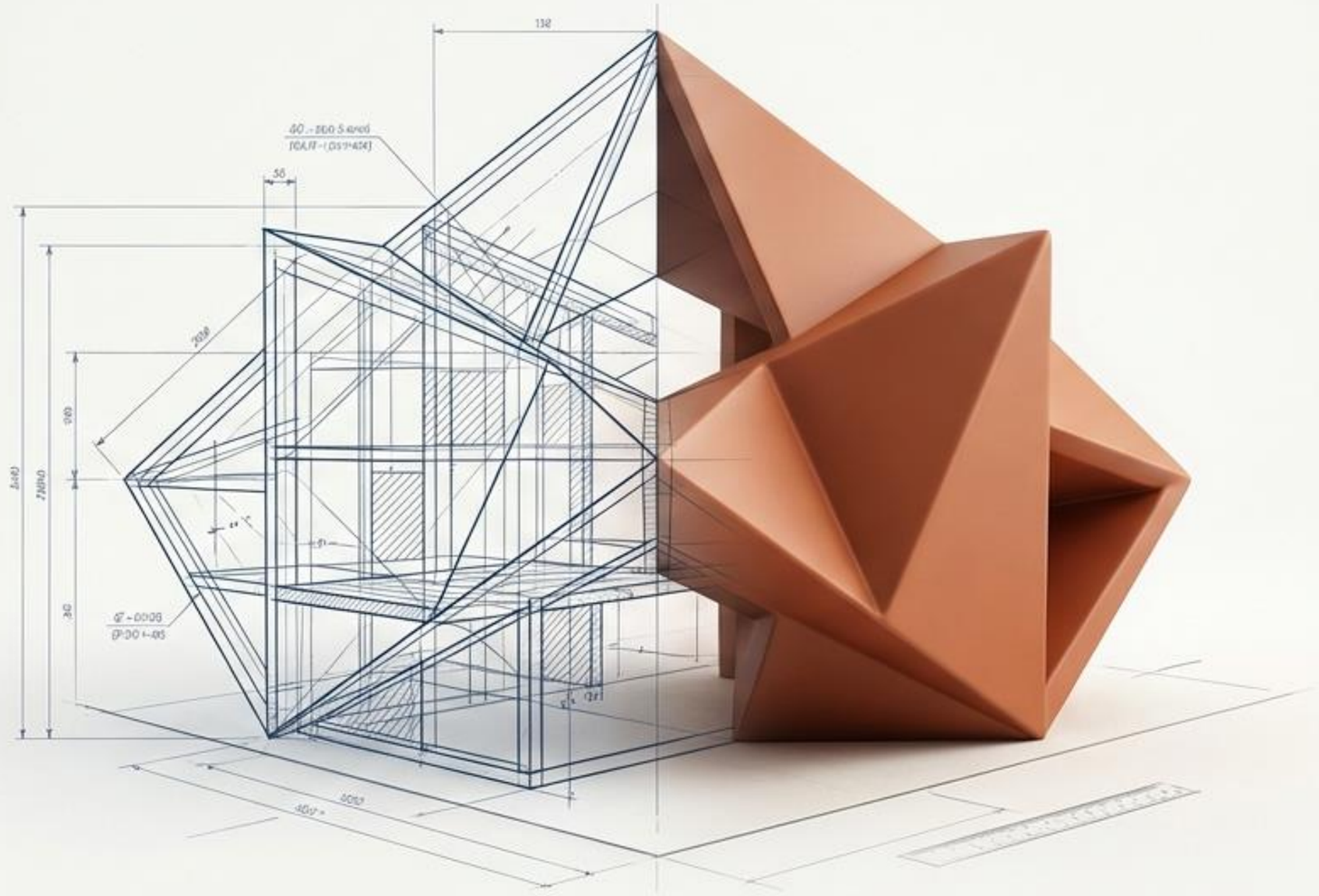


Arsitektur Perangkat Lunak Modern

Penguasaan Pemrograman Berorientasi Objek (OOP) dengan Java

Gemini Notebook

Pergeseran Paradigma: Pembuat Kursi vs. Alat Pertukangan

Pemrograman Prosedural



Fokus utama pada **alat** (fungsi/logika). Data terpisah dari logika, bergerak bebas antar prosedur.

Object-Oriented Programming

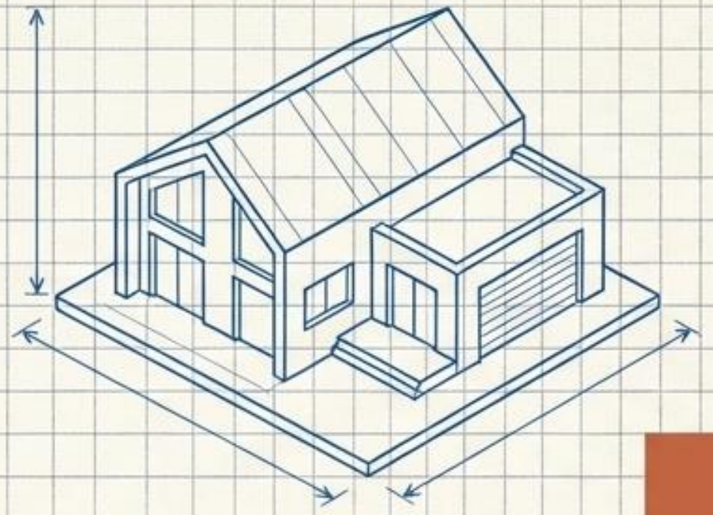


Fokus utama pada **kursi** (objek/data). Data dan antarmuka disatukan dalam satu entitas independen.

OOP mengorganisir kode di sekitar objek di dunia nyata—menggabungkan data dan perilaku untuk membangun sistem yang tangguh dan modular.

Gemini Notebook

Kelas dan Objek: Dari Cetak Biru ke Realita



Kelas (Class): Definisi abstrak.
Cetak biru yang mendeklarasikan atribut dan metode yang akan dimiliki setiap objek.

Instansiasi



Objek (Object): Entitas fisik/logis dalam memori. Setiap objek adalah instance unik dari kelas dengan identitasnya sendiri.

Gemini Notebook

Anatomi Objek: State & Behavior

State / Atribut



Menyimpan informasi tentang seperti apa objek objek tersebut saat ini.

```
String name;
int age;
```

Nilai-nilai ini membentuk State (kondisi) unik objek.

Class Person

Behavior / Metode



Mendefinisikan apa yang dapat dilakukan objek dan bagaimana ia bereaksi.

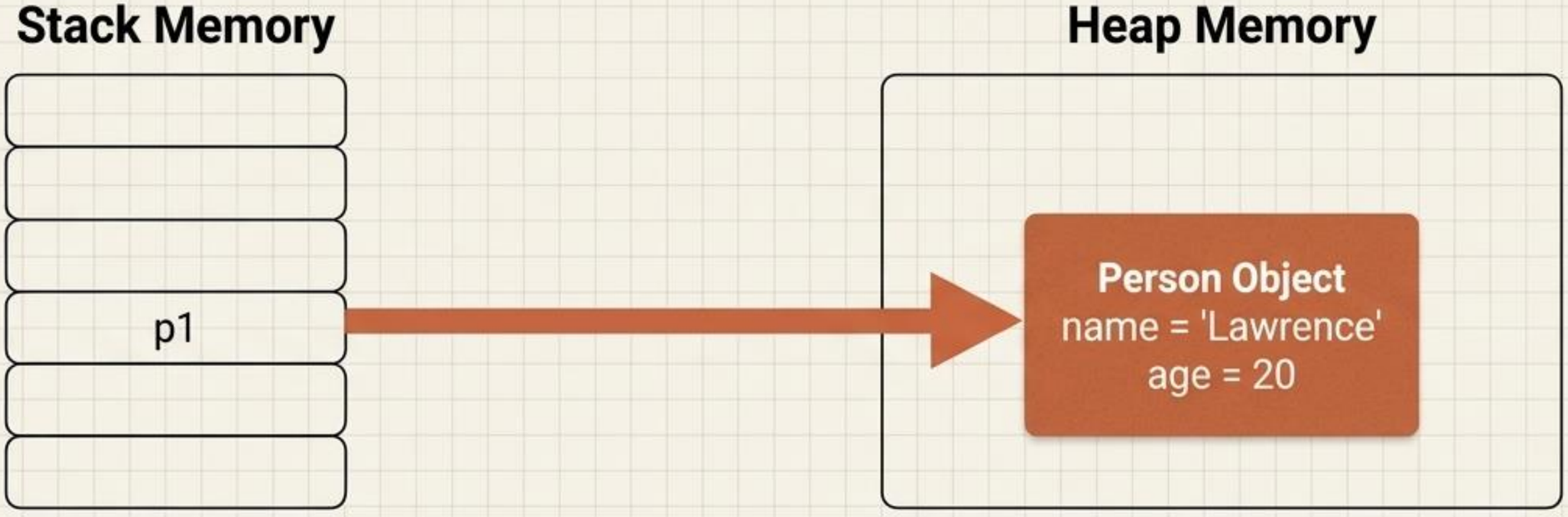
```
showDetails();
setAge();
```

Metode adalah satu-satunya agen yang boleh mengubah State objek.

Gemini Notebook

Arsitektur Memori: Stack vs. Heap

```
Person p1 = new Person("Lawrence", 20);
```



Menyimpan eksekusi metode dan variabel referensi. Variabel p1 hidup di sini, berisi alamat memori.

Tempat alokasi objek dinamis. Objek secara fisik berada di sini dan dikelola oleh Garbage Collector.

Gemini Notebook

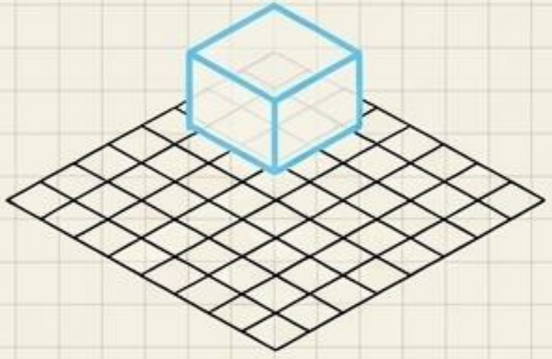
Menghidupkan Objek: Keyword 'new'

1. Deklarasi (Declaration) → **2. Instansiasi (Instantiation)** → **3. Inisialisasi (Initialization)**

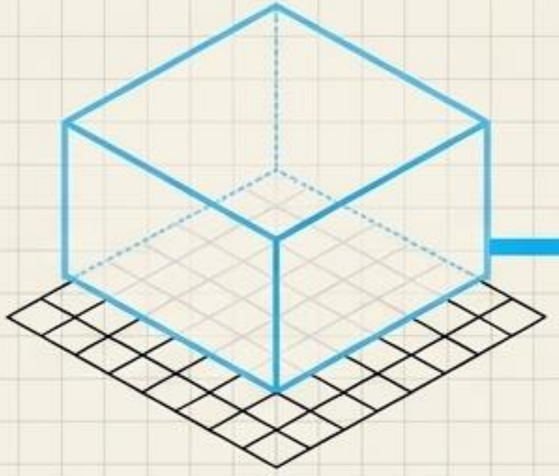
```
Person p1
```

```
new
```

```
Person("Lawrence", 20)
```



Memesan ruang di Stack untuk variabel referensi p1. Nilai awalnya null.



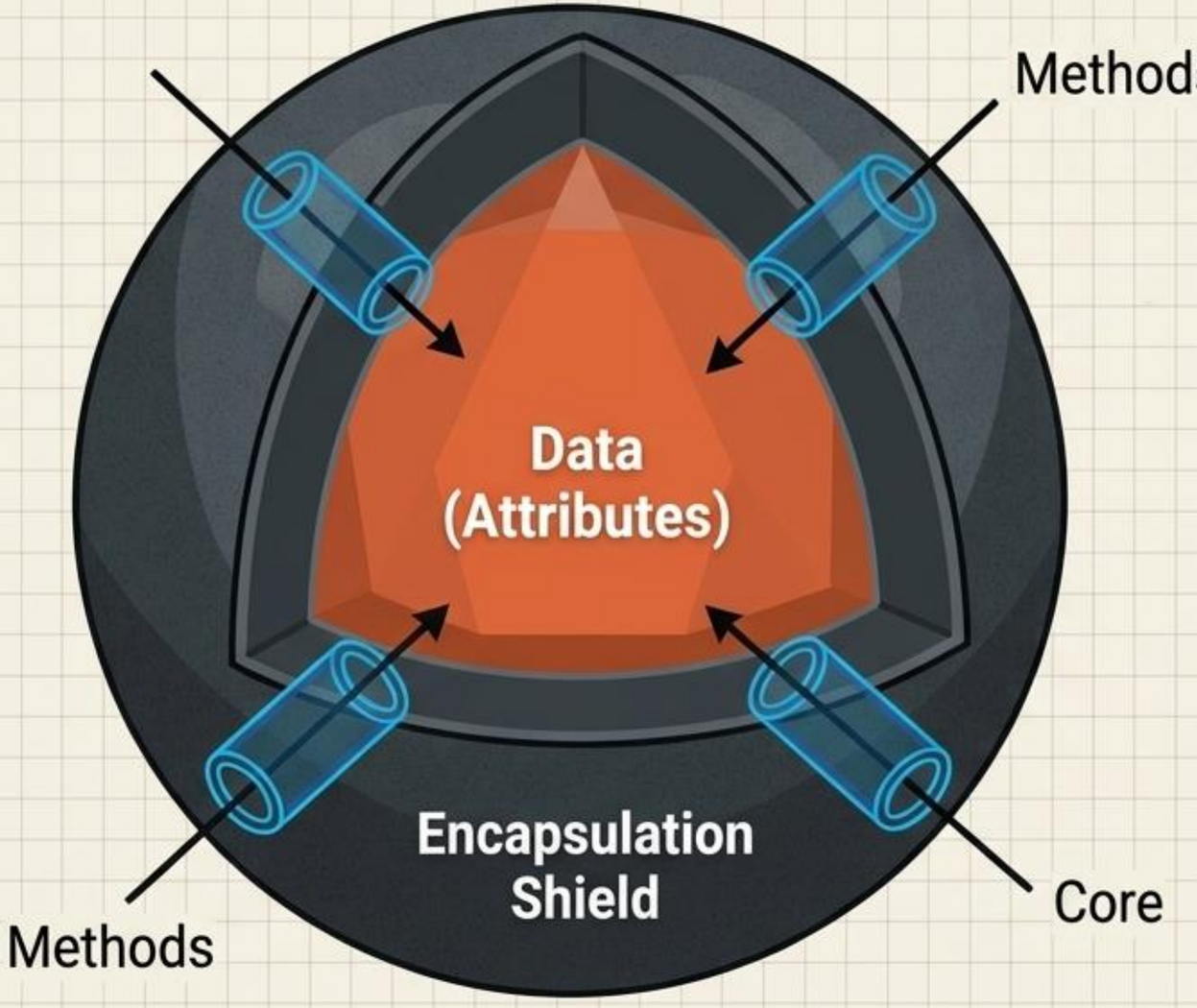
Memesan ruang memori secara dinamis di dalam Heap yang cukup besar untuk menampung satu objek Person.



Memanggil konstruktor untuk menetapkan nilai awal atribut. Menghubungkan alamat Heap ke variabel Stack.

Gemini Notebook

Enkapsulasi: Melindungi Integritas Data



Data Hiding:
Menyembunyikan detail implementasi internal dari pengguna objek (konsep Black Box).

Pemisahan Tanggung Jawab:
Objek lain tidak perlu tahu bagaimana data disimpan, hanya tahu cara berinteraksi dengannya.

Mencegah Anomali:
Melindungi state dari perubahan spontan atau tidak sah dari luar kelas.

Gemini Notebook

Access Modifiers & Mekanisme Getter/Setter

Access Modifiers

private

Hanya bisa diakses dari dalam kelas yang sama (Keamanan tertinggi).

(default)

Bisa diakses dari kelas dalam satu paket.

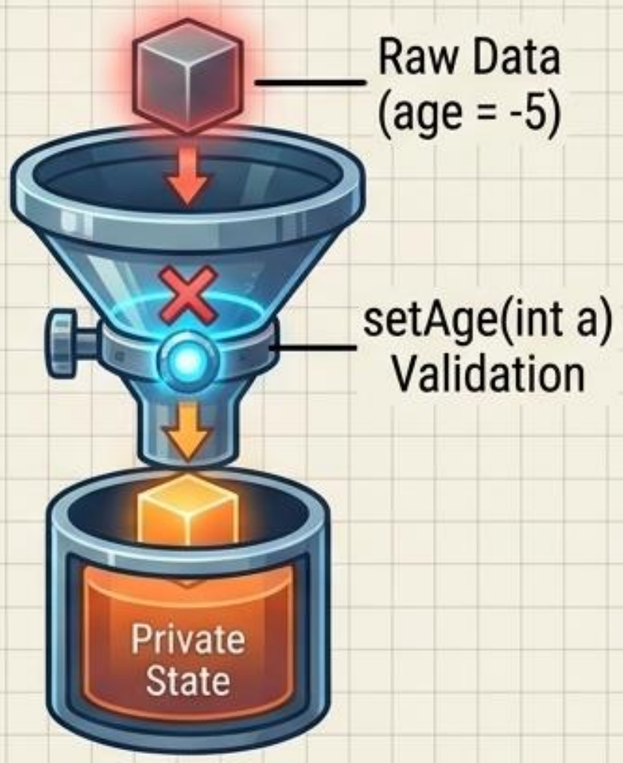
protected

Bisa diakses dari paket yang sama dan oleh subclass.

public

Terbuka untuk seluruh bagian program.

The Gateway - Getter/Setter

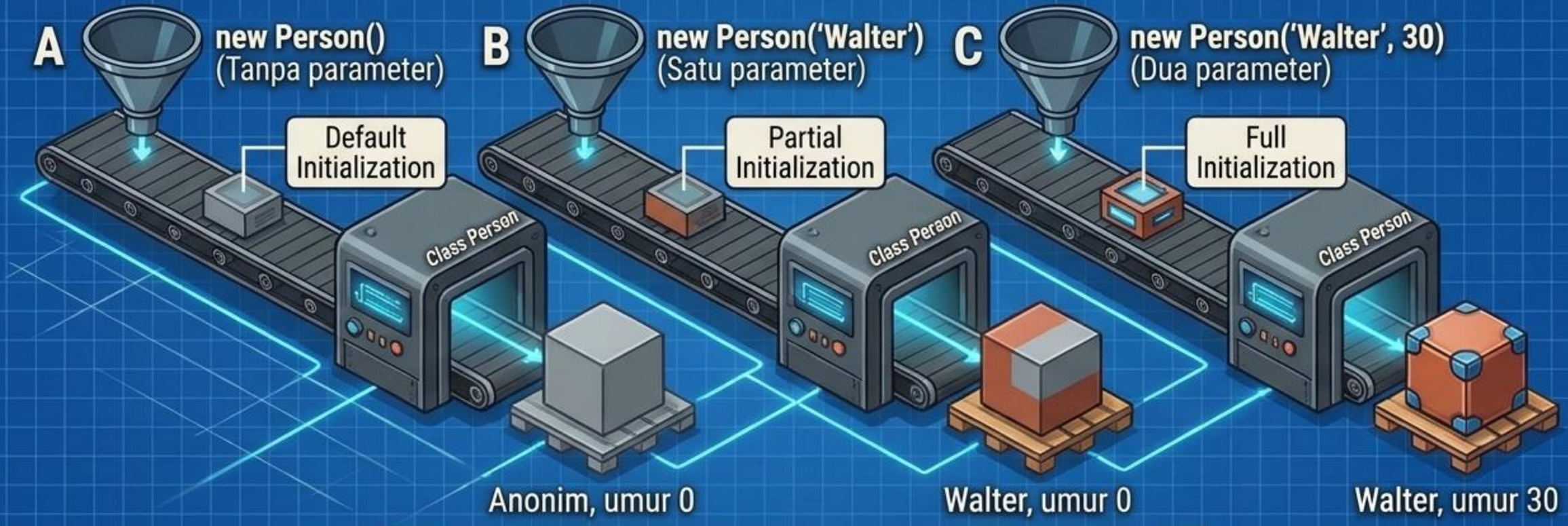


Mengapa tidak **public** saja? Setter bertindak sebagai pos pemeriksaan untuk mencegah data anomali sebelum mengubah atribut.

Gemini Notebook

Konstruktor & Overloading: Variasi Perakitan

Konstruktor adalah metode khusus (nama sama dengan kelas, tanpa **return type**) yang dieksekusi pertama kali saat objek dibuat.

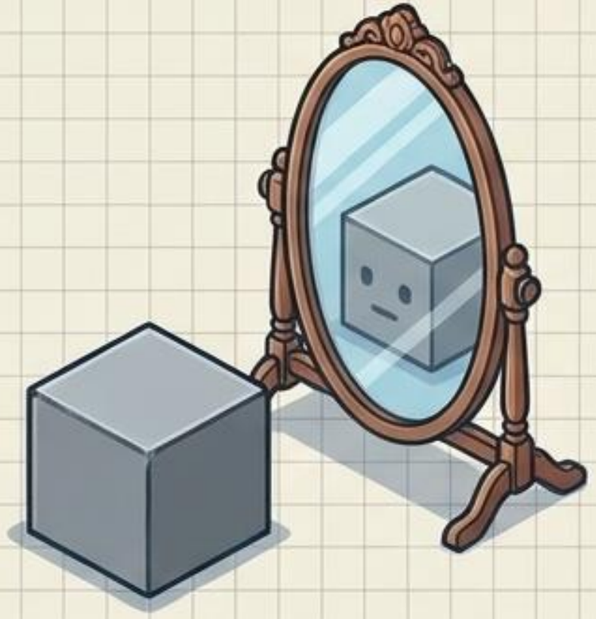


Overloading memungkinkan fleksibilitas instansiasi dengan signature parameter yang berbeda.

Gemini Notebook

Konteks Eksekusi: Keyword ‘this’ dan ‘static’

this (Instance Level)



Referensi ke objek saat ini (current object) yang sedang memanggil metode atau konstruktor.

Menyelesaikan ambiguitas nama variabel (contoh: **this.name** = **name**;) dan meneruskan objek sebagai parameter.

static (Class Level)



Menandakan bahwa field atau metode milik Kelas itu sendiri, bukan milik instance spesifik.

Konstanta global, metode utility (Math.max()), dan data yang dibagikan secara kolektif oleh semua objek dari kelas.

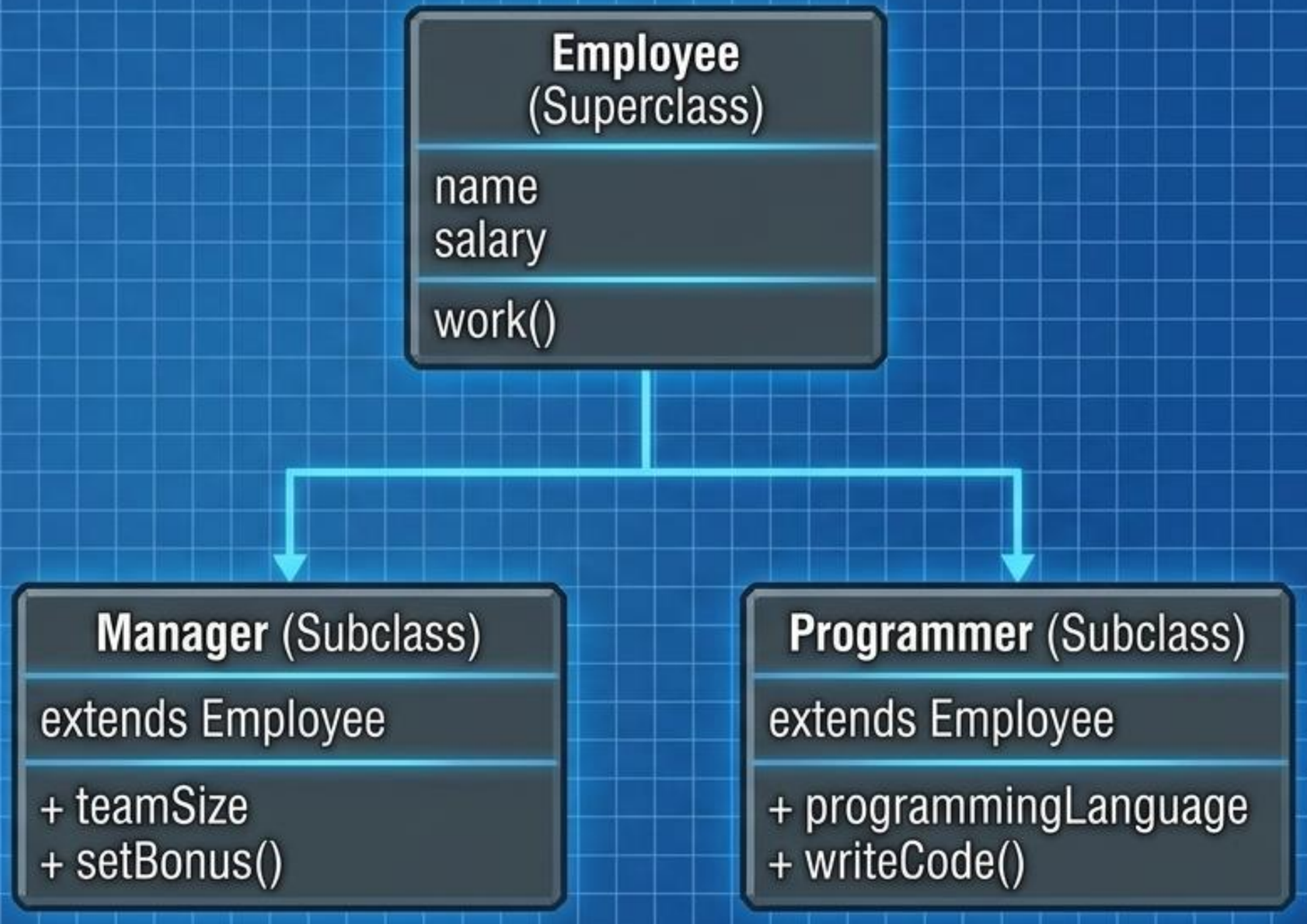
Gemini Notebook

Pewarisan (Inheritance): Hierarki & Reusabilitas

1. **Konsep Dasar**: Mekanisme di mana satu kelas mewarisi state dan behavior dari kelas lain menggunakan keyword ‘extends’.

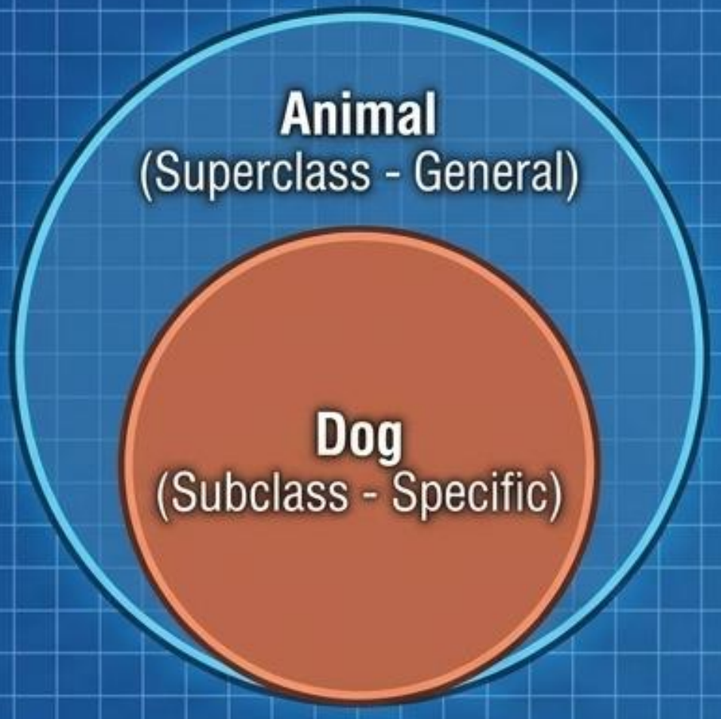
2. **Tujuan Utama**: Reusabilitas kode. Subclass mendapatkan semua metode/atribut non-private secara gratis.

3. **Spesialisasi**: Subclass dapat menambahkan atribut atau metode baru yang tidak dimiliki oleh superclass.



Gemini Notebook

Relasi Superclass & Subclass (‘is-a’)



Aturan Emas Pewarisan: Hubungan ‘Adalah Seekor’ (is-a relationship).

Check 1:
Setiap Dog adalah Animal.

→ ✓

Valid
(Pewarisan diizinkan)

Check 2:
Apakah setiap Animal adalah Dog?

→ ✗

Tidak
(Subclass lebih spesifik)

Catatan Arsitektural: Jika hubungan logis ‘is-a’ tidak terpenuhi, gunakan Komposisi (has-a) alih-alih memaksakan Pewarisan yang salah kaprah.

Gemini Notebook

Evolusi Perilaku: Overriding & Keyword ‘super’

Method Overriding



Subclass menyediakan implementasi baru untuk metode yang sudah ada di Superclass dengan signature yang sama persis.

Keyword ‘super’

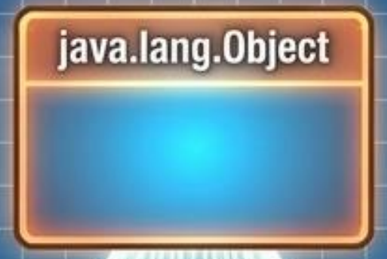


Bagaimana jika kita masih membutuhkan logika dari kelas induk? Keyword ‘super’ digunakan untuk memanggil versi asli metor dari Superclass atau memanggil konstruktor Superclass.

Gemini Notebook

Kelas ‘Object’: Sang Leluhur Kosmis

Di Java, jika sebuah kelas tidak melakukan ‘extends’ pada kelas manapun, secara implisit ia mewarisi ‘Object’. Semua kelas adalah turunan Object.



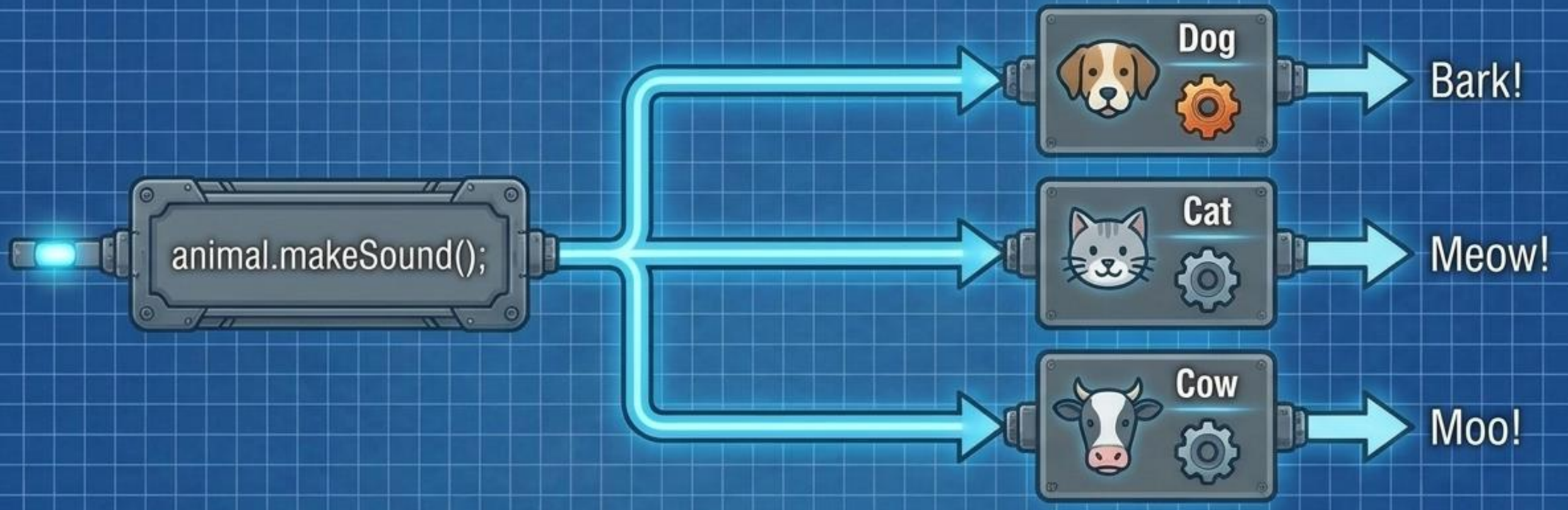
The Big Three Methods

1. **toString():** Mengembalikan representasi string dari objek. Wajib di-override untuk log yang bisa dibaca manusia.
2. **equals(Object obj):** Menguji kesetaraan objek secara logis (bukan sekadar perbandingan alamat memori).
3. **hashCode():** Memberikan identitas numerik unik, krusial untuk struktur data Hash.



Gemini Notebook

Polimorfisme & Dynamic Binding



Konsep Inti: **Polimorfisme** (banyak bentuk) memungkinkan objek merespons panggilan metode yang sama dengan cara mereka sendiri yang unik.

Dynamic Binding: Keputusan tentang implementasi metode mana yang dieksekusi tidak ditentukan saat kompilasi, melainkan saat program berjalan melainkan saat program berjalan (runtime) berdasarkan tipe objek sebenarnya di dalam Heap.

Gemini Notebook

Skala Arsitektur: Heterogeneous Array



Array superclass dapat menampung referensi dari semua tipe subclass-nya.

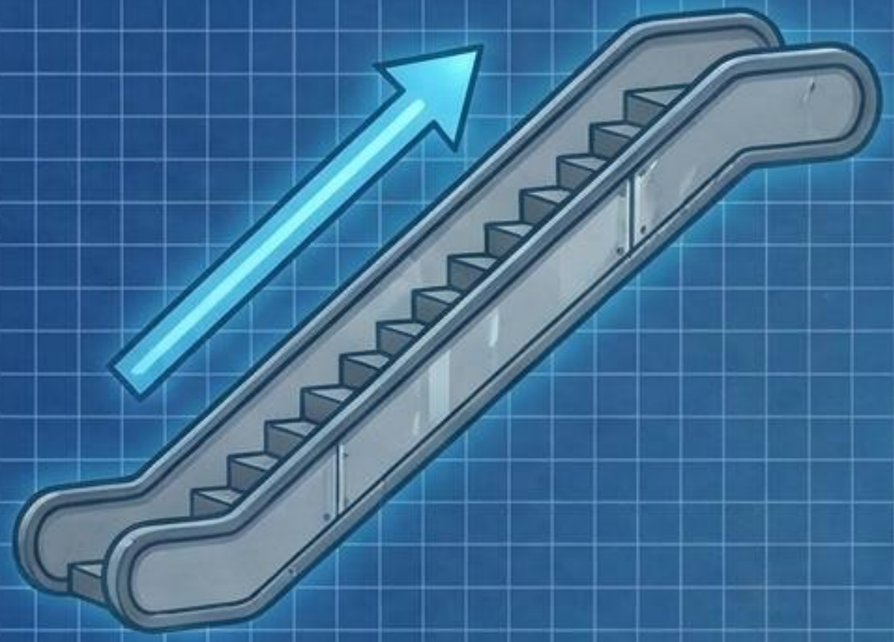


Polymorphic Arguments

Sebuah metode **vetCheck(Animal a)** dapat menerima objek **Dog** maupun **Cat** tanpa harus menulis ulang metode untuk setiap tipe. Ini menghasilkan kode yang generik, fleksibel, dan tahan terhadap penambahan fitur baru di masa depan.

Gemini Notebook

Navigasi Hierarki: Type Casting & ‘instanceof’



Animal a = new Dog();

Upcasting (Aman & Otomatis). Merujuk subclass menggunakan tipe superclass. Fitur spesifik subclass (seperti metode fetch()) disembunyikan dan tidak bisa diakses via referensi ‘a’.



The Safety Net

Operator ‘**instanceof**’ mengecek tipe asli objek di memori sebelum melakukan cast untuk mencegah ClassCastException di runtime.

Dog d = (Dog) a;

Downcasting (Manual & Beresiko).

Mengembalikan referensi ke tipe spesifiknya untuk mengakses fitur khusus.

Gemini Notebook

Abstraksi: Mendefinisikan ‘Apa’ Tanpa ‘Bagaimana’



Abstract Class
(Kumpulan Aturan)

Ditandai dengan keyword ‘abstract’.
Bertindak sebagai templat dasar; tidak dapat diinstansiasi secara langsung (tidak bisa new Animal()).

Concrete Classes
(Implementasi Nyata)

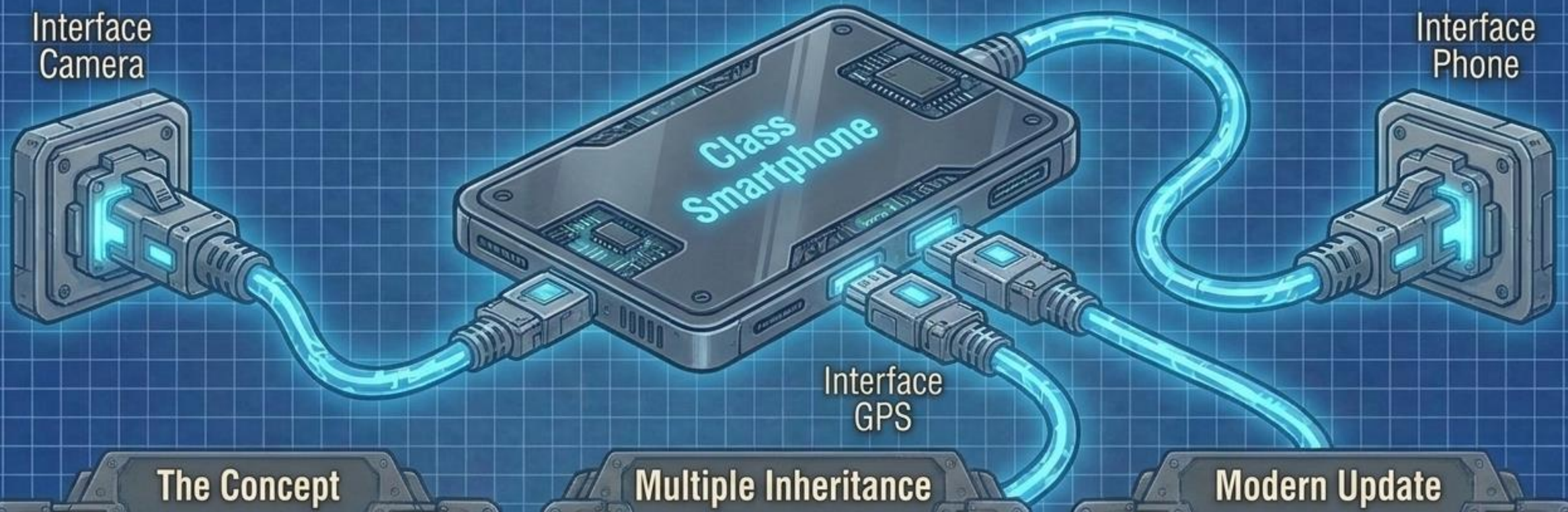


Abstract Methods

Metode tanpa badan (hanya deklarasi: abstract void move();).
Aturan Paksa: Memaksa setiap subclass konkret pertama untuk menyediakan implementasinya sendiri, memastikan standardisasi bentuk.

Gemini Notebook

Antarmuka (Interface) & Multiple Implementation



The Concept

Interface adalah **kontrak murni**. Mendeklarasikan serangkaian metode yang harus dlos yang mengimplementasikannya (menggunakan keyword 'implements').

Multiple Inheritance

Java tidak mengizinkan **pewarisan kelas ganda**. Namun, satu kelas dapat mengimpl banyak antarmuka sekaligus, menciptakan modularitas tinggi.

Modern Update

Sejak Java 8+, Interface dapat memiliki implementasi bawaan menggunakan **Default Methods**, mencegah kerusakan kode lama saat antarmuka diperbarui.

Gemini Notebook

Fitur OOP Java Modern: Singkat & Aman

Records (Data Carriers)



Masalah

Boilerplate berlebihan untuk kelas yang hanya menyimpan data.

Solusi

```
record Point(int x, int y) {}.
```

Otomatis membuat konstruktor, getter, equals, dan toString. Data bersifat immutable.

Enums (Type-Safety)



Konsep

Kelas khusus yang menyimpan kumpulan nilai konstan (type-safe alternative).

Contoh

```
enum Status {
  NEW, IN_PROGRESS, DONE
}
```

Sealed Classes (Control)



Konsep

Membatasi kelas mana saja yang diizinkan menjadi subclass. Memberikan kontrol arsitektural ketat atas hierarki.

Contoh

```
sealed class Shape
  permits Circle, Square
```

Gemini Notebook

Sintesis Arsitektur: Praktik Terbaik Desain OOP

Pemisahan Tanggung Jawab:

Setiap kelas harus memiliki satu fokus atau tanggung jawab yang jelas (Single Responsibility).

Enkapsulasi Maksimal:

Jadikan semua atribut private secara default. Buka akses hanya melalui metode yang teruji.



Komposisi di Atas Pewarisan:
Gunakan relasi ‘has-a’ alih-alih memaksakan hierarki ‘is-a’ yang kaku.

Program ke Antarmuka:

Bergantunglah pada abstraksi (Interface), bukan implementasi konkret, untuk membangun sistem yang adaptif.

Gemini Notebook